LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Noor Khalisa NIM. 2410817220012

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Khalisa
NIM : 2410817220012

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
B. Source Code 6
C. Output Program 14
D. Pembahasan 17
E. Essay ..... **Error! Bookmark not defined.**
F. Daftar Pustaka ..... **Error! Bookmark not defined.**
TAUTAN GIT 26

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose 14
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose 14
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose 15
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML 15
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML 16
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML 16

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt (Compose) 6
Tabel 2. Source Code MainActivity.kt (XML) 9
Tabel 3. Source Code activity_main.xml (XML) 11

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.

B. Source Code

Tabel 1. Source Code MainActivity.kt (Compose)

```
package com.example.tipcalculator

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.ui.res.painterResource
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import java.text.NumberFormat
import java.util.Locale
import kotlin.math.ceil

```

```
22  class MainActivity : ComponentActivity() {
23      override fun onCreate(savedInstanceState: Bundle?) {
24          super.onCreate(savedInstanceState)
25          setContent {
26             MaterialTheme {
27                 Surface(
28                     modifier = Modifier.fillMaxSize(),
29                     color =
    MaterialTheme.colorScheme.background
30                 ) {
31                     TipCalculatorScreen()
32                 }
33             }
34          }
35      }
36  }
37
38  @OptIn(ExperimentalMaterial3Api::class)
39  @Composable
40  fun TipCalculatorScreen() {
41      var amountInput by remember { mutableStateOf("") }
42      var tipPercentage by remember {
    mutableDoubleStateOf(15.0) }
43      var roundUp by remember { mutableStateOf(false) }
44
45      val amount = amountInput.toDoubleOrNull() ?: 0.0
46      val tip = calculateTip(amount, tipPercentage,
    roundUp)
47
48      Column(
49          modifier = Modifier
50              .padding(40.dp)
51              .fillMaxSize(),
52          horizontalAlignment = Alignment.Start,
53          verticalArrangement = Arrangement.Top
54      ) {
55          Text(
56              text = "Calculate Tip",
57              fontSize = 18.sp,
58              modifier = Modifier.padding(bottom = 16.dp)
59          )
60
61          TextField(
62              value = amountInput,
63              onValueChange = {amountInput = it},
64              label = {Text("Bill Amount")},
65              leadingIcon = {
66                  Icon(
67                      painter = painterResource(id =
```

```
R.drawable.money),
                    contentDescription = "Bill Icon"
                )
            },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 24.dp)
        )

        var expanded by remember {mutableStateOf(false)}
        var options = listOf(15.0, 18.0, 20.0)

        ExposedDropdownMenuBox(
            expanded = expanded,
            onExpandedChange = {expanded = !expanded},
            modifier = Modifier.padding(bottom = 24.dp)
        ) {
            TextField(
                value = "${tipPercentage.toInt()}%",
                onValueChange = {},
                readOnly = true,
                label = {Text("Tip Percentage")},
                leadingIcon = {
                    Icon(
                        painter = painterResource(id = R.drawable.percent),
                        contentDescription = "Percent Icon"
                    )
                },
                trailingIcon = {ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded)},
                colors = ExposedDropdownMenuDefaults.textFieldColors(),
                modifier = Modifier.menuAnchor().fillMaxWidth()
            )
            ExposedDropdownMenu(
                expanded = expanded,
                onDismissRequest = {expanded = false}
            ) {
                options.forEach { option -> DropdownMenuItem(
                    text = {Text("${option.toInt()}%")},
```

```
                    onClick = {
                        tipPercentage = option
                        expanded = false
                    }
                )
                }
            }
        }

        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 24.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text(text = "Round up tip?", fontSize = 16.sp)
            Switch(
                checked = roundUp,
                onCheckedChange = {roundUp = it}
            )
        }

        Text(
            text = "Tip Amount: $tip",
            fontSize = 32.sp,
            fontWeight = FontWeight.Bold
        )
    }
}

private fun calculateTip(amount: Double, tipPercentage: Double, roundUp: Boolean): String {
    var tip = (tipPercentage/100) * amount
    if (roundUp) {
        tip = ceil(tip)
    }
    return NumberFormat.getCurrencyInstance(Locale.US).format(tip)
}
```

Tabel 2. Source Code MainActivity.kt (XML)

```
package com.example.tipcalculator

import android.os.Bundle
```

```
import android.text.Editable
import android.text.TextWatcher
import android.widget.ArrayAdapter
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.materialswitch.MaterialSwitch
import com.google.android.material.textfield.TextInputEditText
import java.text.NumberFormat
import java.util.Locale
import kotlin.math.ceil

class MainActivity : AppCompatActivity() {
    private lateinit var etBillAmount: TextInputEditText
    private lateinit var spinnerTipPercentage: com.google.android.material.textfield.MaterialAutoCompleteTextView
    private lateinit var switchRoundUp: MaterialSwitch
    private lateinit var tvTipResult: TextView

    private var currentTipPercentage = 15.0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        etBillAmount = findViewById(R.id.etBillAmount)
        spinnerTipPercentage = findViewById(R.id.spinnerTipPercentage)
        switchRoundUp = findViewById(R.id.switchRoundUp)
        tvTipResult = findViewById(R.id.tvTipResult)

        val tipOptions = arrayOf("15%", "18%", "20%")
        val adapter = ArrayAdapter(this, android.R.layout.simple_dropdown_item_1line, tipOptions)
        spinnerTipPercentage.setAdapter(adapter)

        spinnerTipPercentage.setText(tipOptions[0], false)

        spinnerTipPercentage.setOnItemClickListener {_, _, position, _ ->
            currentTipPercentage = when (position) {
```

```
                0 -> 15.0
                1 -> 18.0
                else -> 20.0
            }
            calculateTip()
        }

        etBillAmount.addTextChangedListener(object : TextWatcher {
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
            override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
            override fun afterTextChanged(s: Editable?) {
                calculateTip()
            }
        })

        switchRoundUp.setOnCheckedChangeListener { _, _ ->
            calculateTip()
        }
    }

    private fun calculateTip() {
        val amount = etBillAmount.text.toString()
        val cost = amount.toDoubleOrNull()

        if (cost == null || cost == 0.0) {
            tvTipResult.text = "Tip Amount: $0.00"
            return
        }

        var tip = (currentTipPercentage / 100) * cost
        if (switchRoundUp.isChecked) {
            tip = ceil(tip)
        }

        val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)
        tvTipResult.text = "Tip Amount: $formattedTip"
    }
}
```

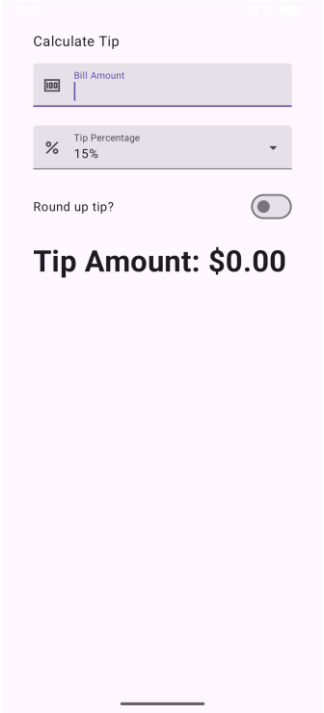
Tabel 3. Source Code activity_main.xml (XML)

```
<?xml version="1.0" encoding="utf-8"?>

```

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/andro
id"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="40dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Calculate Tip"
        android:textSize="17sp"
        android:layout_marginBottom="5dp"/>

<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.Material3.TextInputLayout.FilledBo
x"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="22dp"
        android:hint="Bill Amount"
        app:startIconDrawable="@drawable/money">

<com.google.android.material.textfield.TextInputEditTex
t
            android:id="@+id/etBillAmount"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="numberDecimal"
            android:maxLines="1"/>
</com.google.android.material.textfield.TextInputLayout
>

<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.Material3.TextInputLayout.FilledBo
x.ExposedDropdownMenu"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="32dp"
        android:hint="Tip Percentage"
        app:startIconDrawable="@drawable/percent">
```

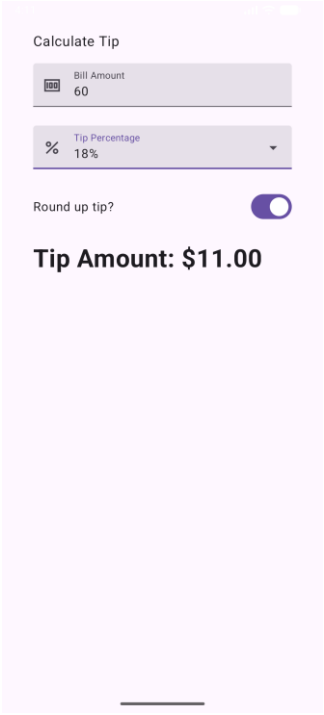
```
39
40
<com.google.android.material.textfield.MaterialAutoComp
41 leteTextView
42             android:id="@+id/spinnerTipPercentage"
43             android:layout_width="match_parent"
44             android:layout_height="wrap_content"
               android:inputType="none" />
45
</com.google.android.material.textfield.TextInputLayout
46 >
47
48     <LinearLayout
49         android:layout_width="match_parent"
50         android:layout_height="wrap_content"
51         android:orientation="horizontal"
52         android:gravity="center_vertical"
53         android:layout_marginTop="16dp"
54         android:layout_marginBottom="16dp">
55
56         <TextView
57             android:layout_width="0dp"
58             android:layout_height="wrap_content"
59             android:layout_weight="1"
60             android:text="Round up tip?"
61             android:textSize="16sp"/>
62
<com.google.android.material.materialswitch.MaterialSwi
63 tch
64             android:id="@+id/switchRoundUp"
65             android:layout_width="wrap_content"
66             android:layout_height="wrap_content"/>
67     </LinearLayout>
68
69     <TextView
70         android:id="@+id/tvTipResult"
71         android:layout_width="wrap_content"
72         android:layout_height="wrap_content"
73         android:text="Tip Amount: $0.00"
74         android:textSize="39sp"
75         android:layout_gravity="center_horizontal"
76         android:textStyle="bold" />
77
</LinearLayout>
```

C. Output Program

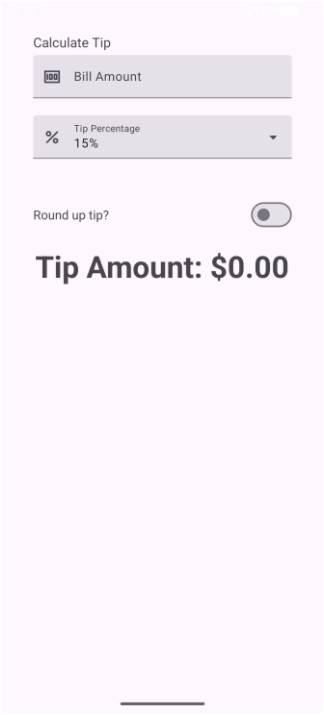


Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose

Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose

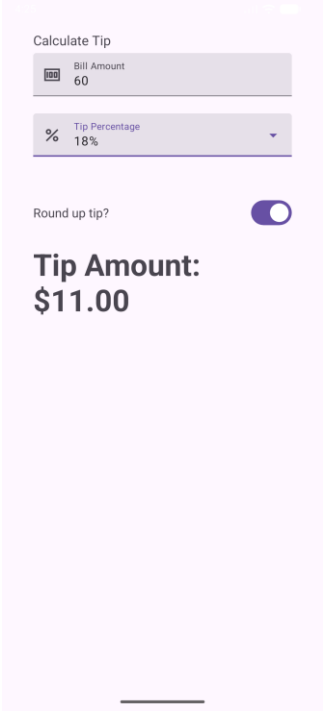


Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 XML

Gambar 5. Screenshot Hasil Jawaban Soal 1 XML



Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

D. Pembahasan

MainActivity.kt (Compose)

Pada baris [1] sampai [20], package digunakan untuk mendeklarasikan direktori file program, sedangkan import digunakan untuk memanggil library dasar Android dan komponen Jetpack Compose (seperti Text, TextField, Column, Row, Switch) yang dibutuhkan untuk merender UI beserta fungsi matematikanya.

Pada baris [22], class 'MainActivity : ComponenActivity()' adalah deklarasi kelas utama yang wajib digunakan pada arsitektur Android modern jika membangun UI menggunakan Jetpack Compose.

Pada baris [23] sampai [25], 'override fun onCreate(…)' adalah metode *lifecycle* utama yang dipanggil saat aplikasi dijalankan, di mana 'setContent {…}' bertindak sebagai blok tempat komponen Composable digambar ke layar.

Pada baris [26] sampai [32], MaterialTheme dan Surface bertindak sebagai latar belakang utama aplikasi, yang kemudian memanggil fungsi 'TipCalculatorScreen()' untuk merender antarmuka kalkulator.

Pada baris [39] dan [40], anotasi @Composable memberitahu kompiler Kotlin bahwa fungsi 'TipCalculatorScreeen()' di bawahnya adalah komponen UI.

Pada baris [41] sampai [43], terdapat mekanisme penyimpanan state bawaan Compose menggunakan remember dan mutableStateOf. Variabel amountInput menyimpan teks tagihan, tipPercentage menyimpan persentase tip, dan roundup menyimpan status *toggle*. Saat nilai-nilai ini berubah, Compose akan otomatis melakukan *recomposition* atau menggambar ulang bagian UI yang terpengaruh.

Pada baris [45] dan [46], variabel amount bertugas mengubah teks input menjadi angka desimal (Double), di mana fungsi 'toDoubleOrNull ?: 0.0' memastikan jika input kosong, nilainya dikembalikan menjadi nol. Kemudian variabel tip memanggil fungsi pembantu 'calculateTip()' untuk mendapatkan hasil perhitungan.

Pada baris [48] sampai [54], tata letak utama dibungkus menggunakan Column agar semua elemen di dalamnya tersusun berurutan dari atas ke bawah.

Pada baris [55] sampai [59], komponen Text digunakan untuk mencetak judul "Calculate Tip".

Pada baris [61] sampai [76], TextField berfungsi sebagai kolom input untuk mengisi nominal tagihan. Kolom ini dilengkapi dengan ikon uang (leadingIcon) dan pengaturan KeyboardType.Number agar papan ketik yang muncul di layar hanya berisi angka.

Pada baris [78] sampai [114], kode berfokus pada pembuatan dropdown menu. Variabel expanded dan options (berisi 15.0, 18.0, 20.0) dibuat untuk mengelola menu tersebut. Elemen ExposedDropdownMenuBox membungkus sebuah TextField yang dibuat readOnly = true dan ExposedDropdownMenu. Ketika menu dibuka dan salah satu opsi pada DropdownMenuItem ditekan (onClick), nilai tipPercentage akan otomatis diperbarui dan menu akan tertutup.

Pada baris [116] sampai [128], elemen Row menyusun komponen secara horizontal dengan gaya Arrangement.SpaceBetween agar posisinya renggan di kiri dan kanan. Di dalamnya terdapat teks "Round up tip?" bersisian dengan komponen Switch. Switch adalah tombol *toggle* yang nilainya terhubung dengan *state* roundup untuk menentukan apakah tip akan dibulatkan atau tidak.

Pada baris [130] sampai [134], Text digunakan Kembali untuk menampilkan hasil akhir perhitungan ($tip) di bagian bawah dengan huruf tebal dan ukuran besar (32.sp).

Pada baris [138] sampai [144], 'private fun calculateTip(…)' adalah fungsi local yang dipisahkan dari UI untuk mengurus logika matematika. Fungsi ini menghitung nominal tip dengan mengalihkan persentase tip dan jumlah tagihan. Jika variabel roundUp bernilai true, fungsi ceil(tip) dari library math akan membulatkan angkanya ke bilangan bulat terdekat di atasnya. Terakhir, angka tersebut diformat ke dalam bentuk mata uang Dolar AS menggunakan fungsi 'NumberFormat.getCurrencyInstance()' sebelum dikembalikan sebagai tampilan teks (String).

MainActivity.kt (XML)

Pada baris [1] sampai [13], package digunakan untuk mendelakrasikan direktori file program, sedangkan import digunakan untuk memanggil library dasar Android dan komponen antarmuka klasik (seperti TextView, TextInputEditText, MaterialSwitch, ArrayAdapter) yang dibutuhkan untuk merender UI beserta fungsi matematikanya.

Pada baris [15], class 'MainActivity : AppCompatActivity()' adalah deklarasi kelas utama yang wajib digunakan pada arsitektur Android untuk merender antarmuka berbasis XML dan menjaga kompatibilitas perangkat.

Pada baris [16] sampai [21], terdapat deklarasi variabel untuk menampung komponen UI menggunakan private lateinit var agar inisialisasinya bisa ditunda hingga tata letak dimuat. Terdapat juga variabel currentTipPercentage untuk menyimpan persentase tip. Berbeda dengan Compose, nilai-nilai ini tidak akan otomatis menggambar ulang UI jika terjadi perubahan.

Pada baris [23] sampai [25], 'override fun onCreate(…)' adalah metode *lifecycle* utama yang dipanggil saat aplikasi dijalankan, di mana setContentView(R.layout.activity_main) bertindak sebagai perintah untuk memuat dan menampilkan desain tata letak XML ke layar.

Pada baris [27] sampai [30], metode findViewById digunakan secara berurutan untuk mencari elemen UI di dalam XML berdasarkan ID-nya, lalu menautkannya ke variabel Kotlin yang sudah dideklarasikan.

Pada baris [32] sampai [45], kode berfokus pada pembuatan dropdown menu. Variabel tipOptions dan adapter dibuat untuk menyuplai daftar pilihan ke spinnerTipPercentage. Ketika menu dibuka dan salah satu opsi ditekan, setOnItemClickListener akan mendeteksi posisinya, memperbarui nilai currentTipPercentage, dan secara manual memanggil fungsi 'calculateTip()'.

Pada baris [47] sampai [53], komponen input tagihan etBillAmount dipasangkan sebuah TextWatcher. Pada blok afterTextChanged, setiap kali pengguna mengetik atau

menghapus angka, system akan mendeteksinya dan langsung memanggil fungsi ‘calculateTip()’.

Pada baris [55] sampai [57], setOnCheckedChangeListener menyusun perintah pemantauan pada tombol *toggle* switchRoundUp. Tombol ini menentukan apakah tip akan dibulatkan atau tidak, dan akan langsung memanggil ‘calculateTip()’ jika nilainya berubah.

Pada baris [60] sampai [77], ‘private fun calculateTip()’ adalah fungsi local yang dipisahkan untuk mengurus logika matematika dan pembaruan layar manual. Variabel amount mengambil teks input dan diubah menjadi angka desimal (Double). Fungsi ini menghitung nominal tip dengan mengalikan persentase dan jumlah tagihan. Jika switchRoundUp.isChecked bernilai true, fungsi ceil(tip) dari library math akan membulatkan angkanya ke bilangan bulat terdekat di atasnya. Terakhir, angka tersebut diformat ke dalam bentuk mata uang Dolar AS menggunakan fungsi ‘NumberFormat.getCurrencyInstance()’ sebelum disisipkan secara eksplisit ke dalam komponen tvTipResult sebagai tampilan teks (String).

activity_main.xml (XML)

Pada baris [1] sampai [7], komponen LinearLayout digunakan sebagai tata letak dasar yang menampung seluruh elemen antarmuka. Atribut xmlns digunakan untuk mendeklarasikan namespace system Android, sementara atribut orientation=”vertical” bertugas menyusun elemen-elemen di dalamnya secara berurutan dari atas ke bawah, dibekali dengan jarak tepi luar (padding) sebesar 40dp.

Pada baris [9] sampai [14], komponen TextView digunakan untuk mencetak teks judul statis “Calculate Tip” dengan ukuran huruf 17sp dan memberikan sedikit jarak ke elemen di bawahnya menggunakan layout_marginBottom.

Pada baris [16] sampai [28], tata letak berfokus pada kolom input tagihan menggunakan TextInput yang membungkus sebuah TextInputEditText. Komponen ini menggunakan gaya visual bawaan Material 3 (FilledBox), dilengkapi dengan ikon uang di sebelah kiri (startIconDrawable), teks petunjuk pengisian (hint), serta pengaturan

inputType=”numberDecimal” agar papan ketik yang muncul di layar hanya memuat angka desimal.

Pada baris [30] sampai [41], komponen TextInputLayout kembali digunakan, namun disetel dengan gaya ExposedDropdownMenu untuk membuat menu pilihan *dropdown*. Komponen ini membungkus sebutah MaterialAutoCompleteTextView yang sengaja diatur dengan inputType=”none” agar pengguna hanya bisa klik dan memilih dari daftar opsi yang disediakan.

Pada baris [43] sampai [58], LinearLayout kedua yang bersarang, disisipkan ke dalam tata letak utama dengan atribut orientation=”horizontal” untuk Menyusun komponen secara menyamping (sama seperti komponen Row di Jetpack Compose). Di dalamnya terdapat TextView yang bersisian dengan komponen MaterialSwitch. Teks tersebut diberi layout_weight=”1” agar teks mengambil seluruh sisa ruang kosong yang tersedia, sehingga secara otomatis mendorong posisi tombol *toggle* ke ujung paling kanan layar.

Pada baris [60] sampai [67], komponen TextView ditempatkan di urutan paling bawah untuk merender hasil perhitungan. Teks default diatur sebagai “Tip Amount: $0.00” dengan ukuran besar (39sp), dicetak tebal, dan diposisikan secara presisi di Tengah layar horizontal menggunakan atribut layout_gravity=”center_horizontal”.

SOAL 2

Soal Praktikum:

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Secara mendasar, perbedaan terbesar antara Jetpack Compose dan XML terletak pada cara pandangnya. Jetpack Compose menggunakan pendekatan deklaratif, artinya mendeskripsikan apa yang ingin ditampilkan berdasarkan kondisi data (*state*) saat ini, dan sistem akan otomatis menggambar ulang setiap kali data tersebut berubah. Semua ini ditulis dalam satu bahasa saja, yaitu Kotlin. Sebaliknya, XML menggunakan pendekatan imperatif, artinya harus memberi instruksi manual langkah demi langkah tentang bagaimana cara menampilkannya. Harus mendesain tata letaknya dulu di file .xml terpisah, kemudian mencari elemen-elemen tersebut di dalam file Kotlin, dan memberikan perintah secara eksplisit kapan UI tersebut harus berubah.

Kelebihannya, Jetpack Compose unggul karena kodenya jauh lebih ringkas dan terpusat, tidak perlu lagi bolak-balik antara file desain dan logika, serta terhindar dari tumpukan kode berulang (*boilerplate*). Manajemen datanya yang otomatis juga menekan risiko *bug* visual. Namun, Compose memiliki kurva pembelajaran yang lumayan tajam karena menuntut *developer* untuk mengubah pola pikir lamanya. Jika konsep *recomposition* tidak dipahami dengan benar, ada risiko aplikasi malah menjadi berat atau lambat.

Di sisi lain, pendekatan XML memiliki keunggulan pada pemisahan tugas yang sangat jelas antara ranah desain visual dan logika program. Karena sudah menjadi standar Android selama belasan tahun, teknologi ini sangat matang, stabil, dan solusi untuk hampir semua *error* bisa ditemukan dengan mudah di internet. Tapi, XML menuntu penulisan kode yang bertele-tele. Pendekatan maualnya juga membuat kode lebih rentan terhadap *bug*, karena seiring bertambah kompleksnya aplikasi, *developer*

sangat gampang lupa untuk memperbarui satu atau dua elemen UI secara manual di layar. Kesimpulannya, meski XML masih sangat bisa diandalkan dan relevan untuk mengurus proyek-proyek lama, Jetpack Compose menawarkan cara kerja yang jauh lebih modern, efisien, dan dinamis, menjadikannya standar masa depan untuk pengembangan aplikasi Android.

ADDITIONAL TASK

Fitur Unggulan Kotlin

1. *Null Safety*

   Kotlin dirancang untuk membasmi NullPointerException atau yang sering disebut "The Billion Dollar Mistake". Secara bawaan, variabel di Kotlin tidak bisa bernilai null kecuali secara eksplisit mengizinkannya dengan menambahkan tanda ?.
2. Interoperabilitas dengan Java

   Kotlin dan Java bisa beroperasi berdampingan di proyek yang sama secara mulus. Kode Java bisa dipanggil dari Kotlin, begitu pula sebalikna, sehingga transisi dari *codebase* lama sangat mudah.
3. *Coroutines*

   Fitur tingkat lanjut untuk menangani proses *asynchronous* seperti mengambil data dari API atau *database*. *Coroutines* sangat ringan dan tidak memblokir antarmuka pengguna (*main thread*), sehingga aplikasi terhindar dari *freeze* atau *lag*.
4. *Extension Functions*

   Memungkinkan untuk menambahkan fungsi baru ke dalam suatu *class*, bahkan *class* bawaan sistem seperti String atau komponen UI tanpa perlu mewarisi atau mengubah *class* tersebut.
5. *Smart Casts*

Kompilator Kotlin pintar dalam melacak tipe data. Jika sudah mengecek tipe suatu objek menggunakan keyword is, kompilator akan otomatis mengonversi (*casting*) objek tersebut di baris kode selanjutnya.

6. *Scope Function* (let, run, with, apply, also)

   Fitur ini erat kaitannya dengan praktik *chaining* (menyambung operasi) yang elegan. Fungsi ini memungkinkan eksekusi sekumpulan kode secara langsung di dalam kontes sebuah objek, membuat strukutr kode menjadi jauh lebih ringkas.

7. Concisness

   Kotlin memangkas sangat banyak *boilerplate code*. Misalnya pada Data Class, kompilator otomatis membuatkan fungsi *getter*, *setter*, 'equals()', 'hashCode()', dan 'toString()' hanya dengan satu kata kunci penanda.

Test Case (Penggunaan Null Safety & Scope Function)

Pada test case ini, diilustrasikan simulasi pemrosesan data pengguna yang mungkin bernilai null.

```
fun main() {
    var nameNonNull: String = "Noor"
    var nameNullable: String? = null

    val lengthOfName: Int? = nameNullable?.length
    println("Panjang nama (Safe Call): $lengthOfName")

    val safeLength = nameNullable?.length ?: 0
    println("Panjang nama (Elvis Operator): $safeLength")

    nameNullable = "Khalisa"

    nameNullable?.let { name ->
        println("Data ditemukan, mari diproses: $name")
        println("Panjang karakternya adalah: ${name.length}")
    }
}
```

'fun main()' adalah fungsi utama yang menjadi titik awal pengeksekusian program Kotlin.

Terdapat deklarasi dua variabel dengan pendekatan *Null Safety* yang berbeda. Variabel nameNonNull secara tegas dideklarasikan agar hanya bisa diisi oleh teks (String) dan tidak boleh kosong, sedangkan nameNullable dibekali dengan tanda tanya (?) pada tipe datanya (String?) yang mengizinkannya untuk menampung nilai kosong atau null.

Variabel lengthOfName menggunakan operator *Safe Call* (?.) untuk mencoba mengakses panjang karakter. Karena nameNullable saat ini bernilai null, operasi ini tidak akan membuat aplikasi *crash* (terhindar dari NullPointerException), melainkan secara aman hanya akan mengembalikan nilai null untuk dicetak ke layar.

Variabel safeLength mengimplementasikan *Elvis Operator* (?:). Operator ini bertugas memberikan nilai Cadangan, jika hasil pengecekan panjang karakter di sebelah kirinya bernilai null, maka sistem akan otomatis menggunakan angka 0 (yang berada di sebelah kanannya) sebagai nilai pengganti mutlaknya.

Kemudian variabel nameNullable diperbarui nilainya dari yang semula kosong (null) menjadi sebuah teks String yang valid, yaitu “Khalisa”.

Lalu, fitur *Scope Function* berupa let digabungkan dengan operator *Safe Call* (?.let). Blok kode di dalam kurung kurawal bertindak sebagai pelindung ekstra yang hanya akan dieksekusi jika nameNullable sudah tidak bernilai null. Di dalam blok ini, data yang sudah divalidasi tersebut diwakili oleh variabel name, sehingga data tersebut bebas dieksekusi dengan aman tanpa perlu lagi menambahkan pelindung tanda tanya untuk mencetak isinya beserta panjang karakternya.

DAFTAR PUSTAKA

[1] S. Aigner, R. Elizarov, S. Isakova dan D. Jemerov, Kotlin in Action, Second Edition, Manning Publications, 2024.

TAUTAN GIT

https://github.com/lisaryuna/Praktikum-Mobile